

FinReportGen

Automated Financial-Analysis Report Generator — DCF, Comps, Risk Analysis

Tech Stack: Next.js · Fastify · Python · yFinance · Claude API

Version v1.0 · June 2026 · Internal Engineering Specification

Table of Contents

#	Section	Description
1	Project Overview	Goals, scope, report types, stakeholders
2	System Architecture	Frontend, backend, AI layer, data sources
3	Data Ingestion Layer	yFinance integration, caching, normalisation
4	Financial Modelling Engine	DCF, comps, risk — formulas & assumptions
5	AI Narrative Layer	Claude API integration, prompt design, guardrails
6	Report Generation	PDF assembly, template system, branding
7	Frontend — Next.js	UI components, report viewer, input forms
8	Backend API — Fastify	REST routes, auth, queue, webhooks
9	Database & Caching	Schema, Redis caching strategy
10	Security & Compliance	Data handling, API key management, audit log

1. Project Overview

FinReportGen takes raw financial statements (uploaded as Excel/CSV or fetched via yFinance) and automatically produces a full equity research report including a DCF valuation, comparable company analysis, and risk section — with AI-generated narrative via the Claude API.

Languages	TypeScript (Next.js 14, Fastify), Python 3.11 (modelling engine)
AI Layer	Anthropic Claude API (claude-sonnet-4-6)
Data Sources	yFinance, SEC EDGAR, manual upload (Excel/CSV)
Output Formats	PDF (primary), JSON (API), XLSX (data export)
Auth	NextAuth.js (Google OAuth + email/magic-link)

1.1 Report Types Supported

- **Full Equity Research Report:** DCF + comps + risk + AI narrative (~15–25 pages)
- **Quick Valuation Snapshot:** DCF only with 1-page summary
- **Peer Benchmarking Report:** Comps table for user-defined peer group
- **Risk Dashboard:** Beta, VaR, scenario analysis, covenant summary

2. System Architecture

Layer	Technology	Role
Frontend	Next.js 14 (App Router), Tailwind, shadcn/ui	Report configuration UI, PDF viewer, dashboard
API Gateway	Fastify 4 (Node.js)	REST API, auth middleware, job queue dispatch
Modelling Engine	Python (FastAPI microservice)	DCF, comps, risk calculations
AI Narrative	Claude API (Anthropic)	Generates analyst-style prose from model outputs
Data Layer	yFinance, SEC EDGAR, Redis	Market/financial data, caching
Report Builder	WeasyPrint / Puppeteer	HTML→PDF with custom template
Database	PostgreSQL 16	Users, report history, model inputs/outputs

3. Data Ingestion Layer

3.1 yFinance Wrapper

```
// yFinance caching wrapper
# data/yfinance_client.py
class FinancialDataClient:
    CACHE_TTL = 3600 # 1 hour for market data

    async def get_financials(self, ticker: str) -> CompanyFinancials:
        cached = await self.redis.get(f'financials:{ticker}')
        if cached: return CompanyFinancials.parse_raw(cached)

        yf_ticker = yf.Ticker(ticker)
        data = CompanyFinancials(
            income_stmt = yf_ticker.income_stmt,
            balance_sheet = yf_ticker.balance_sheet,
            cash_flow = yf_ticker.cashflow,
            info = yf_ticker.info,
        )
        await self.redis.setex(f'financials:{ticker}', self.CACHE_TTL, data.json())
        return data
```

3.2 Manual Upload — Normalisation Rules

- Accepted formats: **.xlsx** (preferred), **.csv**, **.json**
- Required sheets/columns validated against **financial_schema.json**; missing fields prompt user to map columns
- All monetary values normalised to USD millions; FX rates fetched from ECB API at upload timestamp
- Negative convention enforced: expenses always positive, net income can be negative
- LTM (Last Twelve Months) automatically calculated from quarterly data if annual not present

4. Financial Modelling Engine

4.1 DCF Model

The DCF module implements a 2-stage model: explicit 5-year forecast + terminal value.

```
// DCF engine (simplified)
# models/dcf.py
def run_dcf(inputs: DCFInputs) -> DCFOutput:
    fcfs = project_free_cash_flows(
        base_revenue=inputs.ltm_revenue,
        growth_rates=inputs.growth_rates, # [g1..g5]
        ebitda_margins=inputs.ebitda_margins,
        capex_pct=inputs.capex_pct,
        nwc_change_pct=inputs.nwc_change_pct,
    )
    terminal_value = fcfs[-1] * (1 + inputs.tgr) / (inputs.wacc - inputs.tgr)
    discount_factors = [(1/(1+inputs.wacc))**t for t in range(1, 6)]
    pv_fcfs = sum(fcf * df for fcf, df in zip(fcfs, discount_factors))
    pv_tv = terminal_value * discount_factors[-1]
    enterprise_value = pv_fcfs + pv_tv
    equity_value = enterprise_value - inputs.net_debt
    price_target = equity_value / inputs.shares_outstanding
    return DCFOutput(ev=enterprise_value, equity_value=equity_value,
        price_target=price_target, pv_fcfs=pv_fcfs, pv_tv=pv_tv)
```

4.2 Comparable Company Analysis (Comps)

- Peer group sourced from user input or auto-suggested by sector/SIC code
- Multiples calculated: EV/EBITDA, EV/Revenue, P/E, P/FCF, EV/EBIT
- 25th / median / 75th percentile computed across peer group
- Subject company implied valuation range derived from peer multiple range × subject metric
- Football-field chart data prepared for frontend rendering

4.3 Risk Module

- **Beta:** OLS regression of weekly returns vs SPY, 2-year window; re-levered using Hamada equation
- **VaR (95%, 99%):** Historical simulation + parametric (normal) methods; 1-day and 10-day horizons
- **Scenario Analysis:** Base / Bull / Bear cases with user-adjustable macro assumptions
- **Sensitivity Tables:** 2-way tables for WACC vs TGR (DCF) and EV/EBITDA vs EBITDA (comps)

5. AI Narrative Layer

FinReportGen uses the **Claude API** to generate analyst-quality prose for each report section. Model outputs from the financial engine are serialised to structured JSON and injected into carefully designed prompts.

5.1 Prompt Architecture

- **System prompt:** Sets persona as senior equity research analyst; instructs on tone, disclaimers, formatting conventions
- **Section prompts:** One API call per major section (executive summary, valuation, risks, investment thesis)
- **Model:** *claude-sonnet-4-6* for standard reports; *claude-opus-4-6* for premium tier
- **Max tokens:** 1,200 per section to control report length
- **Temperature:** 0.3 for factual valuation sections; 0.6 for investment thesis

5.2 Guardrails

- All AI-generated text prefixed with disclosure: *'This content is AI-generated and not investment advice.'*
- Factual claims extracted from model outputs are cross-checked against source data; mismatches flagged for human review
- No specific buy/sell/hold recommendation generated without explicit user opt-in
- Hallucination mitigation: prompts instruct model to use only data provided in context, not training knowledge

6. Report Generation

6.1 Report Assembly Pipeline

- **Step 1:** Modelling engine returns JSON output (DCF, comps, risk metrics)
- **Step 2:** Claude API called for each narrative section
- **Step 3:** Jinja2 HTML templates populated with data + narrative
- **Step 4:** Puppeteer (headless Chrome) renders HTML → PDF with precise pagination
- **Step 5:** PDF uploaded to S3; signed URL returned to user

6.2 Report Structure

Section	Source	Pages
Cover Page	Template	1
Executive Summary	AI (Claude)	1–2
Company Overview	AI + yFinance data	1–2
Financial Summary	Engine + tables	2–3
DCF Valuation	Engine + AI commentary	3–4
Comparable Analysis	Engine + AI commentary	2–3
Risk Analysis	Engine + AI commentary	2–3
Investment Thesis	AI (Claude)	1
Disclaimers	Template (static)	1

7. Frontend — Next.js

7.1 Key Routes

- **/dashboard**: List of generated reports, usage stats, quick actions
- **/report/new**: Multi-step wizard — ticker search → peer selection → assumption inputs → generate
- **/report/[id]**: Embedded PDF viewer (react-pdf) + model assumption editor
- **/report/[id]/data**: Raw data tables with export to XLSX
- **/settings**: API key management, branding customisation, billing

7.2 State Management

- Server state: **TanStack Query** for report list, status polling, financial data
- Form state: **React Hook Form + Zod** for multi-step assumption inputs
- Report generation status: **Server-Sent Events** from Fastify for real-time progress updates

8. Backend API — Fastify

Method	Route	Description
POST	/api/reports	Create new report generation job
GET	/api/reports	List reports for authenticated user
GET	/api/reports/:id/status	Poll generation status (SSE endpoint)
GET	/api/reports/:id/download	Get signed S3 download URL
GET	/api/financials/:ticker	Fetch and cache financial data
POST	/api/financials/upload	Upload manual financial statement file
GET	/api/peers/suggest/:ticker	AI-suggested peer group for ticker

9. Database & Caching

```
// Core PostgreSQL schema
-- reports: tracks each generation job
CREATE TABLE reports (
  id UUID PRIMARY KEY, user_id UUID NOT NULL,
  ticker TEXT, peer_tickers TEXT[], status TEXT, -- queued|running|done|failed
  model_inputs JSONB, model_outputs JSONB,
  pdf_s3_key TEXT, created_at TIMESTAMPTZ DEFAULT NOW(),
  completed_at TIMESTAMPTZ
);

-- report_sections: AI narrative per section
CREATE TABLE report_sections (
  id UUID PRIMARY KEY, report_id UUID REFERENCES reports(id),
  section_name TEXT, content TEXT,
  prompt_tokens INT, completion_tokens INT, model TEXT
);
```

9.1 Redis Caching Strategy

- **Financial data:** TTL 1 hour (market data changes infrequently during market hours)
- **Peer suggestions:** TTL 24 hours
- **Report status:** TTL 10 min (after completion, DB is source of truth)
- **Rate-limit counters:** TTL 60 seconds (sliding window per user/IP)

10. Security & Compliance

10.1 API Key Management

- Anthropic API keys stored in **AWS Secrets Manager**; never logged or exposed to frontend
- Per-user API key rotation supported; current key version tracked in **users.api_key_version**
- Request signing: Fastify validates JWT on every protected route using **@fastify/jwt**

10.2 Data Handling

- Uploaded financial statements stored in S3 with server-side encryption (AES-256)
- User-uploaded data automatically deleted after 90 days unless report saved
- No financial data shared with third parties beyond necessary API calls
- Audit log: every report generation, model call, and download event recorded in **audit_events** table

10.3 Regulatory Disclaimer

■■ *FinReportGen outputs are for informational purposes only. All generated reports must include the standard financial disclaimer. The platform is not registered as an investment adviser. Consult legal counsel before commercialising this tool.*